Fil rouge FR-PN – Réseau de Petri

Conception du simulateur : diagramme de classes et diagrammes de séquence

Aslam et Arrsène

IMT Atlantique – Informatique MAPD – année 2026-2027

1. Présentation

Ce document présente la conception retenue pour le simulateur de réseau de Petri. Il contient le diagramme de classes détaillé retenu (section 2), deux diagrammes de séquence du tir d'une transition, dans le cas simple et dans le cas général (section 3), et les justifications de nos principaux choix, appuyées par des extraits de variantes de diagrammes de classes (section 4).

Le modèle couvre les 10 exigences de l'énoncé et l'extension de l'exercice 2 (arcs zéro et arcs videurs). **Convention de nommage :** un arc est dit *entrant* ou *sortant* du point de vue de la transition (*ArcEntrant* : Place vers Transition ; *ArcSortant* : Transition vers Place), ce qui est l'inverse de l'énoncé, qui se place du point de vue de la place.

Principales décisions

- Les arcs sont typés par héritage (*ArcEntrant*, *ArcSortant*, puis *ArcZero* et *ArcVideur*) et le tir repose sur le polymorphisme, sans test de type.
- Chaque arc porte le couple (*place*, *transition*) qu'il relie ; le réseau garde la liste de toutes les transitions.
- La logique métier est répartie : la place gère ses jetons, l'arc sait comment agir sur sa place, la transition sait si elle est franchissable, le réseau choisit.
- *ReseauPetri* est la classe principale : elle réifie le réseau et porte directement les opérations de construction (ajout et suppression d'éléments) et d'exécution.

2. Diagramme de classes détaillé retenu

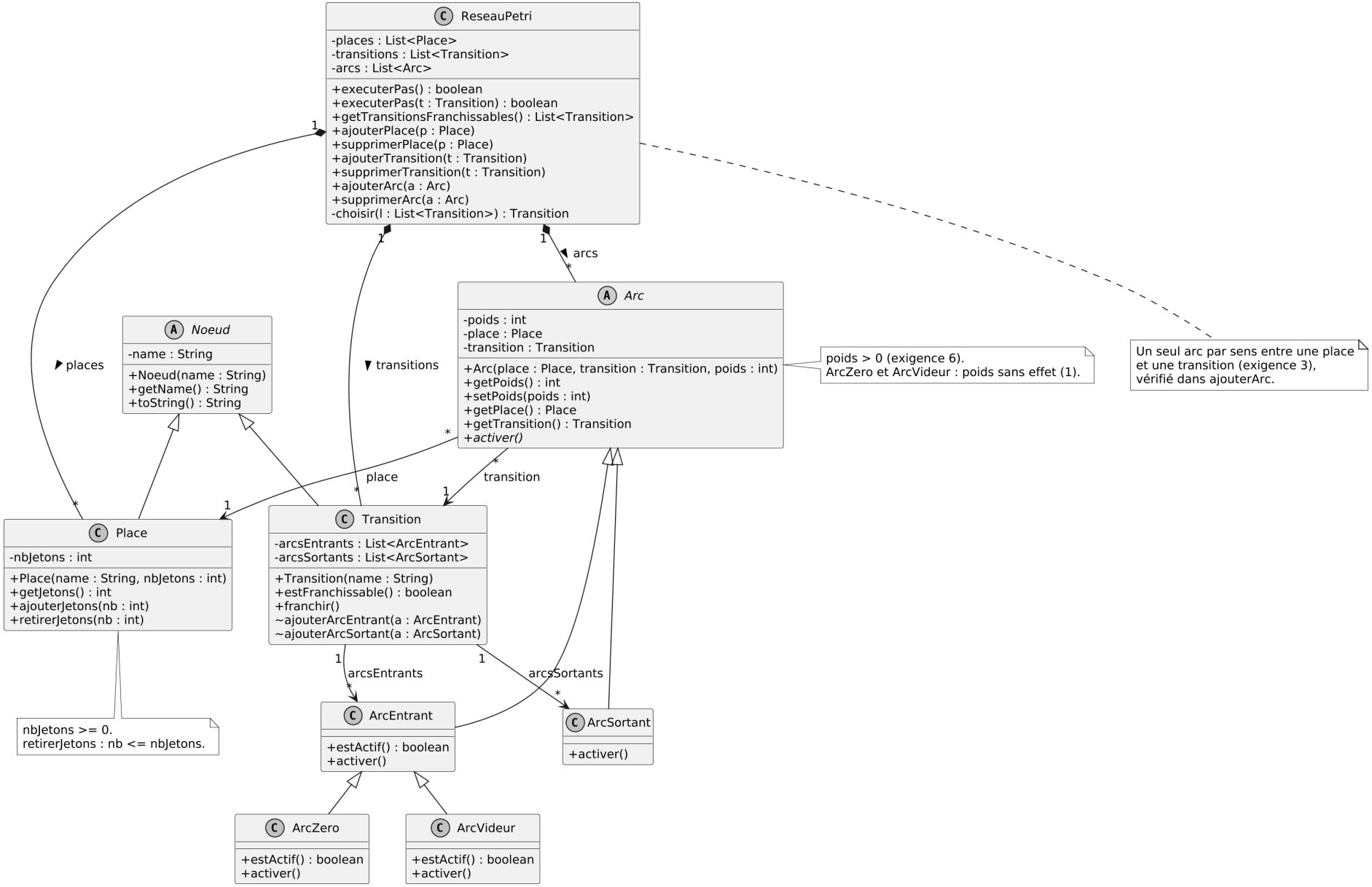


Figure 1 – Diagramme de classes retenu (les cardinalités, rôles et notes de contraintes sont portés sur le diagramme).

2.1 Rôle de chaque classe

Classe	Responsabilité
ReseauPetri	Classe principale et racine du modèle : possède les places, transitions et arcs, permet de les ajouter et de les supprimer. Exécute un pas : *executerPas()* tire une transition choisie au hasard parmi les franchissables, *executerPas(t)* tire celle désignée par l'utilisateur.
Noeud (abstraite)	Porte le nom, commun à *Place* et *Transition*.
Place	Gère son nombre de jetons (ajout, retrait, lecture).
Transition	Connaît ses arcs entrants et sortants ; décide si elle est franchissable (*estFranchissable*) et se tire (*franchir*).
Arc (abstraite)	Porte le poids et le couple (place, transition) ; *activer()* est abstraite.
ArcEntrant	Place vers Transition. Actif si la place contient au moins *poids* jetons ; *activer()* retire *poids* jetons.
ArcSortant	Transition vers Place. *activer()* ajoute *poids* jetons.
ArcZero	Spécialise *ArcEntrant* : actif seulement si la place est vide ; *activer()* ne retire rien.
ArcVideur	Spécialise *ArcEntrant* : actif dès qu'il y a un jeton ; *activer()* retire tous les jetons.

2.2 Couverture des exigences

Exigence	Où elle est traitée
1. Réseau = places, transitions, arcs	*ReseauPetri* (listes *places*, *transitions*, *arcs*)
2. Arcs Place-Transition et Transition-Place	*Arc* relie une *Place* et une *Transition* ; le sens est porté par *ArcEntrant* / *ArcSortant*
3. Un seul arc par sens entre une place et une transition	Note UML ; vérification dans *ajouterArc*
4. Poids de l'arc	*Arc.poids*, utilisé dans *activer()* et *estActif()*
5. L'utilisateur choisit la transition à tirer	*executerPas(t : Transition)*
6. Poids strictement positif	Note UML ; contrôle dans le constructeur et dans *setPoids*
7. Place vide ou avec jetons	*Place.nbJetons* >= 0 (note UML)
8. Ajouter / supprimer places, transitions, arcs	*ReseauPetri* (*ajouterPlace*, *supprimerArc*, etc.)
9. Ajouter / enlever des jetons	*Place.ajouterJetons*, *Place.retirerJetons*
10. Changer la valeur d'un arc	*Arc.setPoids*

3. Diagrammes de séquence

3.1 Cas simple : une transition avec un seul arc entrant et un seul arc sortant

L'utilisateur désigne la transition *t* (*executerPas(t)*). Le réseau demande à *t* si elle est franchissable ; *t* interroge son arc entrant, qui lit les jetons de la place d'entrée. Si elle l'est, *franchir()* active l'arc entrant (retrait de *poids* jetons) puis l'arc sortant (ajout de *poids* jetons). Le résultat est rendu au client (*true* / *false*).

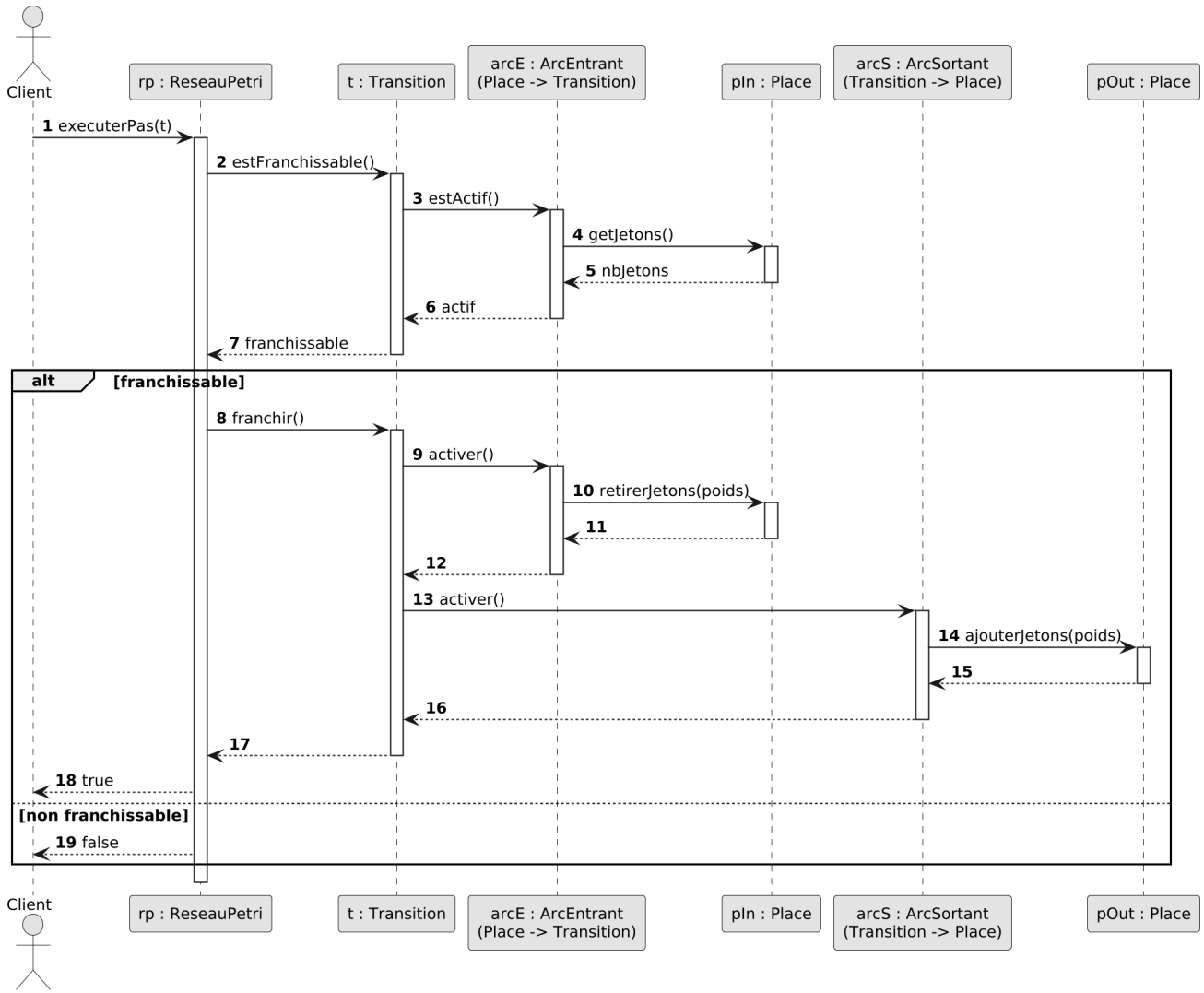


Figure 2 – Tir d'une transition désignée, cas simple.

3.2 Cas général : plusieurs arcs entrants et sortants, choix non déterministe

executerPas() commence par construire la liste des transitions franchissables : une transition l'est si tous ses arcs entrants sont actifs. Une transition sans arc entrant est donc toujours franchissable. Si la liste n'est pas vide, le réseau en choisit une (*choisir*, non déterministe), la tire, et rend *true* ; sinon il rend *false*. Les notes montrent comment *ArcZero* et *ArcVideur* redéfinissent le comportement sans changer le déroulement.

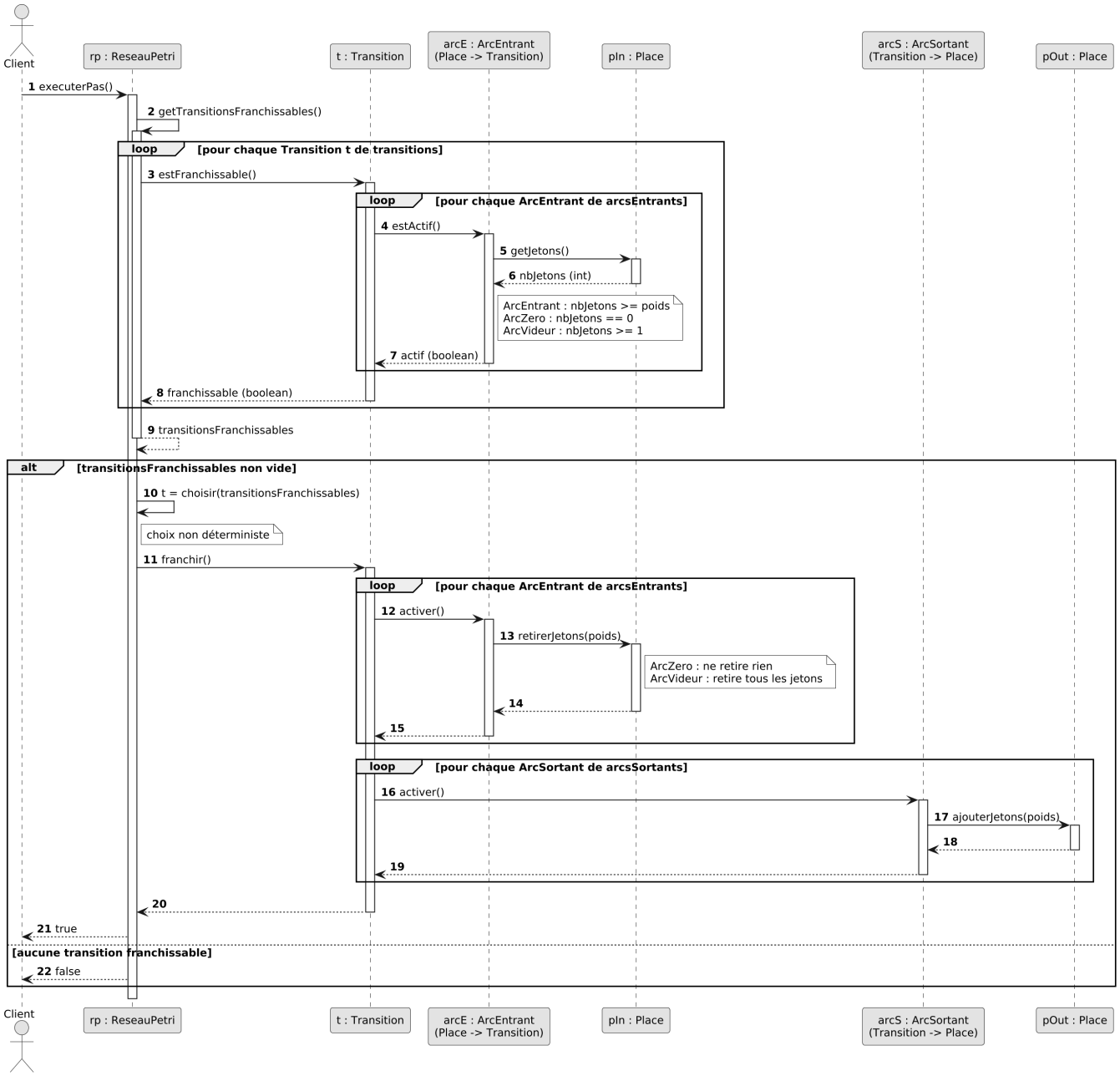


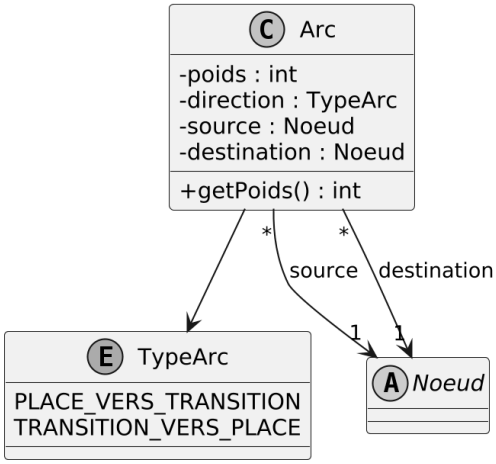
Figure 3 – Pas de simulation, cas général.

4. Justification des principaux choix

4.1 Modéliser les arcs : trois variantes

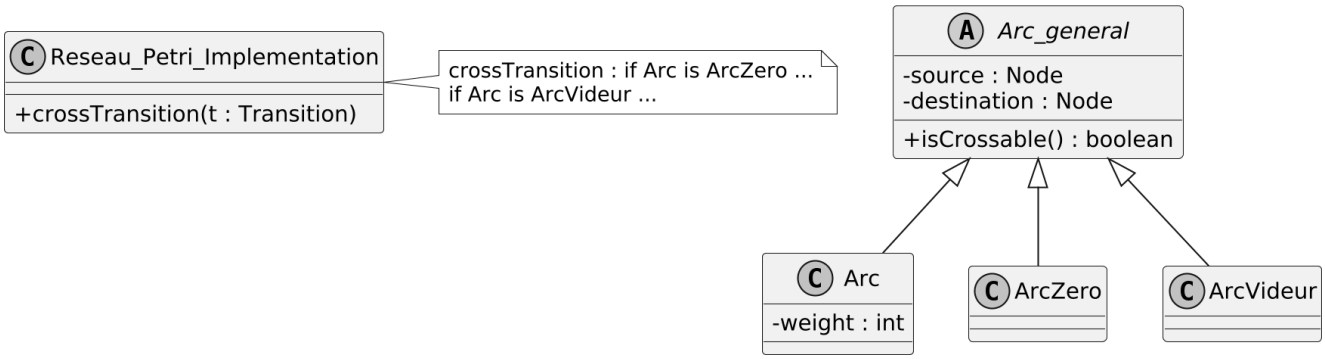
La première variante (A) est la plus directe : une seule classe *Arc* avec une énumération donnant le sens. La variante B est celle d'un modèle comparé lors de la relecture croisée : une classe mère d'arcs entre deux *Node* quelconques et des sous-classes pour les arcs spéciaux. Nous avons retenu la variante C.

Variante A : un seul type d'arc + énumération



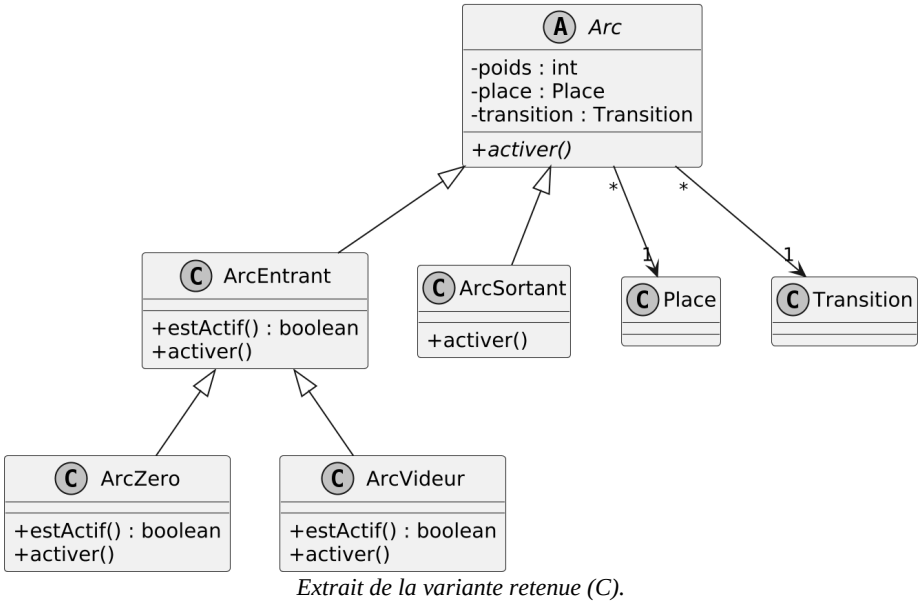
Extrait de la variante A.

Variante B (modèle comparé) : arcs entre Node, tests de type dans le réseau



Extrait de la variante B (modèle comparé).

Variante retenue : arcs typés, polymorphisme



Extrait de la variante retenue (C).

	Avantages	Inconvénients
A. *Arc* + *TypeArc*	Une seule classe, simple à lire.	Le sens doit être testé partout (*if direction* == ...) ; extrémités typées *Noeud* : un arc Place-Place ou Transition-Transition reste possible (exigence 2) et il faut caster pour atteindre la place ; chaque nouveau type d'arc modifie le code existant.
B. Arcs entre *Node*, tests de type dans le réseau	Hiérarchie d'arcs ; *isCrossable()* redéfini.	Mêmes défauts d'extrémités ; le réseau teste le type des arcs (*if Arc is ArcZero*) et doit donc connaître chaque sous-classe ; seule la franchissabilité est polymorphe, pas le tir. Responsabilités concentrées dans le réseau (Po2, P4).
C. Arcs typés + polymorphisme (retenue)	Exigence 2 garantie par le typage ; aucun cast ; ajouter un type d'arc = ajouter une sous-classe sans toucher à *Transition* ni à *ReseauPetri* ; les arcs zéro et videur sont déjà prêts.	Plus de classes ; deux listes dans *Transition* ; poids sans sens pour *ArcZero* et *ArcVideur* (fixé à 1).

4.2 Le couple (place, transition) porté par chaque arc

Chaque arc connaît sa place et sa transition. Cela garantit par construction qu'un arc relie bien une place et une transition, évite les casts, et permet de retrouver les arcs d'une transition à sa suppression. Le réseau garde en plus la liste de toutes les transitions, nécessaire pour calculer les transitions franchissables.

4.3 Navigabilité des associations

Association	Choix	Justification
ReseauPetri vers *Place*, *Transition*, *Arc*	Composition, navigable du réseau vers ses éléments	Le réseau possède ses éléments et en contrôle la durée de vie ; il parcourt les transitions pour le pas.
Arc vers *Place*	Unidirectionnelle	L'arc agit sur sa place ; la place n'a pas à connaître ses arcs. Couplage faible : *Place* reste simple et réutilisable.
Transition vers *ArcEntrant* / *ArcSortant*	Navigable, deux listes distinctes	La transition parcourt ses arcs pour *estFranchissable* et *franchir* ; deux listes typées évitent tout test de type.
Arc vers *Transition*	Navigable (donc bidirectionnelle avec la précédente)	Redondance volontaire : l'arc sait à quelle transition il appartient (suppression, cohérence). Elle est maintenue par *ReseauPetri.ajouterArc*.

La liste *ReseauPetri.arcs* est aussi redondante (on pourrait la déduire des transitions). Nous la gardons pour parcourir, afficher et supprimer les arcs sans passer par les transitions. Le prix est une copie de plus à maintenir : c'est pourquoi *ReseauPetri* est le seul point de modification.

4.4 Répartition des responsabilités et encapsulation

- Le tir n'est pas codé dans le réseau : la transition coordonne, chaque arc sait agir sur sa place. *Transition* et les arcs ne sont donc pas de simples structures de données (Po6).
- *Place* n'expose pas de *setJetons* : seuls *ajouterJetons* et *retirerJetons* existent, ce qui protège l'invariant *nbJetons* >= 0.
- *ajouterArcEntrant* et *ajouterArcSortant* ont une visibilité paquet (~) : seul *ReseauPetri.ajouterArc* les appelle, ce qui empêche de désynchroniser *Arc.transition* et les listes de la transition (C3).
- Tous les attributs sont privés ; *choisir* est privée car c'est un détail du pas.

4.5 Redéfinitions et surcharges

- **Redéfinitions :** *activer()* est abstraite dans *Arc*, réalisée dans *ArcEntrant* et *ArcSortant* ; *ArcZero* et *ArcVideur* redéfinissent *estActif()* et *activer()* d'*ArcEntrant*. Ce sont les méthodes à redéfinir pour l'extension des arcs.

- **Surcharge :** *executerPas()* (choix automatique) et *executerPas(t)* (choix de l'utilisateur, exigence 5).

4.6 Réification du réseau, sans interface de service

ReseauPetri réifie la notion de réseau : c'est la classe principale, qui possède places, transitions et arcs et offre directement les opérations de construction et d'exécution. Nous n'avons pas ajouté d'interface de service (type *GestionReseau*) : il n'y a pour l'instant qu'une seule réalisation, et l'interface ajouterait une classe sans gain immédiat (O1, C1). Le réseau étant le seul point de modification, une interface pourra être extraite plus tard si une seconde réalisation devient utile, par exemple derrière un adaptateur lors de l'intégration (P10, Pj4).

4.7 Noeud, nom et identité

Noeud factorise le nom des places et des transitions. Le nom est fixé à la construction et ne change pas ; il sert à l'affichage, au *toString()* et aux tests. Nous n'avons pas gardé d'*id* séparé : il ferait doublon avec le nom, imposerait de vérifier l'unicité, et les références d'objets suffisent à identifier les éléments. L'égalité reste l'égalité par référence.

4.8 Contraintes non exprimables en UML

Poids strictement positif, un seul arc par sens entre une place et une transition, nombre de jetons positif : ces règles sont notées sur le diagramme et seront vérifiées à l'entrée du code (constructeur d'*Arc* et *setPoids*, *ajouterArc*, précondition de *retirerJetons*). *executerPas(t)* rend *false* si la transition n'est pas franchissable, sans modifier le réseau.

4.9 Limite connue

Le poids n'a pas de sens pour *ArcZero* et *ArcVideur* mais il est hérité d'*Arc* ; nous le fixons à 1. Une variante plus fine sortirait le poids de la racine, au prix d'une hiérarchie plus lourde ; nous avons préféré la simplicité (O1, C1).